Jsp - Servlet

Chapitre 1: Premier pas avec les Servlets

Dr Ahmad Khoureich Ka

UADB

L3 D2AW

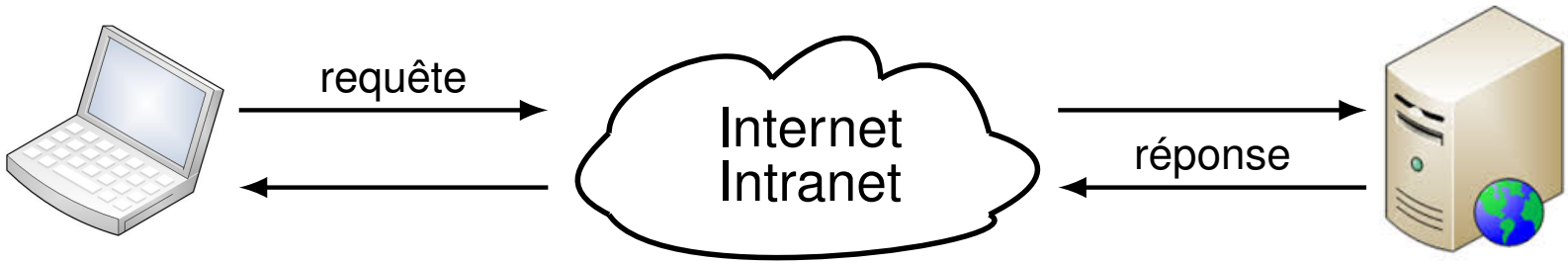
1. Application web
   - C'est quoi une application web ?
2. Le servlet
   - Qu'est-ce que c'est
   - Fonctionnement
   - Cycle de vie
   - Structure de base
   - Exemples
3. Structure d'une App web
   - Structure des répertoires
   - Le fichier context.xml
   - Le fichier web.xml
   - Les annotations

Application web

- C'est une application qui suit le modèle client-serveur.
- Elle est exécutée par un logiciel serveur sur un ordinateur ou un terminal mobile (le serveur).
- L'accès à l'application web se fait depuis un logiciel client exécuté sur un ordinateur ou un terminal mobile (le client).
- La communication entre le client et le serveur se fait selon le protocole HTTP à travers un réseau (internet ou intranet).
- Le serveur peut communiquer en même temps avec plusieurs clients.

Application web

- Le client (généralement un navigateur web)
  - envoie une requête
  - reçoit et présente la réponse
- Le serveur HTTP (logiciel)
  - reçoit les requêtes
  - produit et envoie les réponses



Avantages des applications web

- Mise à jour et maintenance sans distribution ni installation de logiciel chez le client.
- Les sauvegardes sont centralisées sur le serveur.
- Compatibilité multi-plateformes (Composants matériels) et multi-systèmes (Composants logiciels).
- Disponible partout (pour tous), à tout moment.

Remarque :

- La sécurité des données est gérée de manière centralisée.

Le servlet : Qu'est-ce que c'est

- Le servlet assure le coté dynamique d'une application web JavaEE.
- Un servlet est une classe Java qui implémente des méthodes de traitement de requêtes et éventuellement de génération de réponses.
- Une fois compilé et chargé, le servlet est associé à une url et s'exécute coté serveur dans un conteneur de servlets.

Le servlet : Avantages

- **Facile à utiliser pour :**
  - la lecture des données de formulaires
  - la lecture et la modification des en-têtes HTTP
  - le transfert de requêtes
  - la gestion des cookies et de la session
  - etc.
- **Documentation :** disponibilité d'une documentation très fournie sur les classes, interfaces et paquets de la plate-forme Java EE.
- **Framework :** Plusieurs frameworks sont disponibles pour faciliter et accélérer le processus de développement. Exemple : JSF, Struts, Hibernate, Spring MVC, etc.
- **Portable et sûre :** écrit en Java
- **Pas chère :** logiciels serveurs et outils de développement gratuits.

Le servlet : Fonctionnement

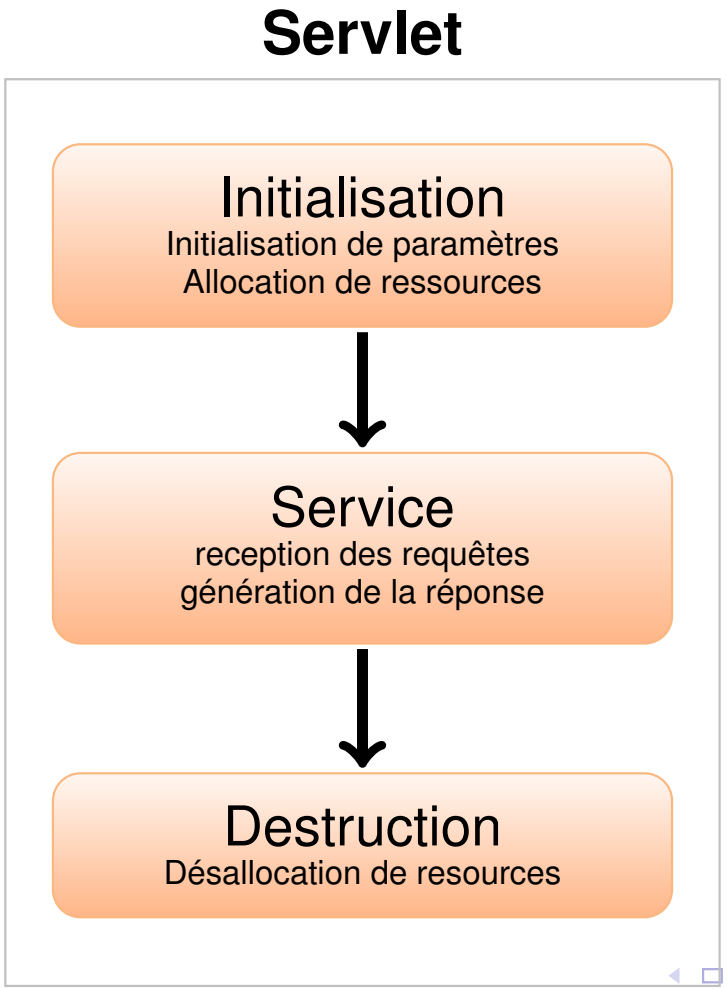
Lorsque le serveur reçoit une requête, il charge (si ce n'est pas encore fait) le servlet qui doit la traiter.

Le servlet effectue les actions suivantes :

1. lire la requête envoyée par le client (données de formulaires et en-têtes HTTP de requête) ;
2. générer la réponse (codes de statut, en-têtes HTTP de réponse, texte, html, images, etc.) ;
3. envoyer la réponse vers le client.

Le servlet : Cycle de vie

- Un servlet étend la classe abstraite **`javax.servlet.http.HttpServlet`**



Le servlet : Cycle de vie

Initialisation :

- Méthode : `init()` ou `init(ServletConfig config)`.
- Exécuter une seule fois lors du chargement du servlet.
- Utile pour initialiser certains paramètres.

Le servlet : Cycle de vie

Mise en service :

- Méthode : `service()`
- Exécutée à chaque fois qu'une requête est reçue
- Suivant la nature de la requête, la méthode `service()` délègue le traitement de la requête à :
  - `doGet(HttpServletRequest request, HttpServletResponse response)`

    ou
  - `doPost(HttpServletRequest request, HttpServletResponse response)`
- `HttpServletRequest` représente la requête reçue
- `HttpServletResponse` représente la réponse à envoyer au client

Le servlet : Cycle de vie

Destruction :

- Méthode `destroy()`
- Exécutée une seule fois à l'arrêt du servlet
- Utile pour libérer les ressources allouées

Les méthodes `init()` de HttpServlet

- Il y'a 2 versions de la méthode `init()` héritées de `javax.servlet.GenericServlet` :
  1. celle qui est appelée par le conteneur de servlet avant la mise en service du servlet

```
public void init(ServletConfig config) throws ServletException {
  super.init(config); //Obligatoire
  ...
}
```

- Remarque : l'appel `super.init(config)` doit être la première instruction de cette version de `init`.
- L'objet `config` dispose de la méthode :
  `String getInitParameter(String name)`
  permettant de récupérer la valeur d'un paramètre d'initialisation du servlet.

Les méthodes `init()` de HttpServlet

2. celle qui est appelée par la première version

```
public void init() throws ServletException {
  ...
}
```

- ☞ Remarque : Cette version de `init` ne nécessite pas l'appel `super.init(config);`.
- ☞ Remarque : L'objet `ServletConfig` peut être obtenu avec la méthode `getServletConfig()` de `javax.servlet.GenericServlet`.

Structure de base du Servlet

- Un servlet doit étendre la classe abstraite `javax.servlet.http.HttpServlet`

- Sa structure de base (sous Netbeans) est composée des méthodes :
  1. `processRequest(HttpServletRequest request, HttpServletResponse response)` qui traite la requête et génére la réponse.
  2. `doGet(HttpServletRequest request, HttpServletResponse response)` qui appelle la méthode **`processRequest`**
  3. `doPost(HttpServletRequest request, HttpServletResponse response)` qui appelle la méthode **`processRequest`**

Exemple 1 : Servlet qui répond avec du texte

```
package textservlet;
import java.io.*;
import javax.servlet.http.*;
import javax.servlet.ServletException;

public class TextServlet extends HttpServlet {

   protected void processRequest(HttpServletRequest request,
       HttpServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text");

      try(PrintWriter out = response.getWriter()) {
         out.println("Nom: <input type='text' name='nom'/><br/>");
      }
   }
```

Exemple 1 : Servlet qui répond avec du texte (suite)

```
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
          throws ServletException, IOException {
       processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
          throws ServletException, IOException {
       processRequest(request, response);
    }
}
```

Exemple 2 : Servlet qui répond avec du html

```
package htmlservlet;
import java.io.*;
import javax.servlet.http.*;
import javax.servlet.ServletException;

public class HtmlServlet extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest request,
       HttpServletResponse response)
         throws ServletException, IOException {
      processRequest(request, response);
   }
   @Override
   protected void doPost(HttpServletRequest request,
       HttpServletResponse response)
         throws ServletException, IOException {
      processRequest(request, response);
   }
```

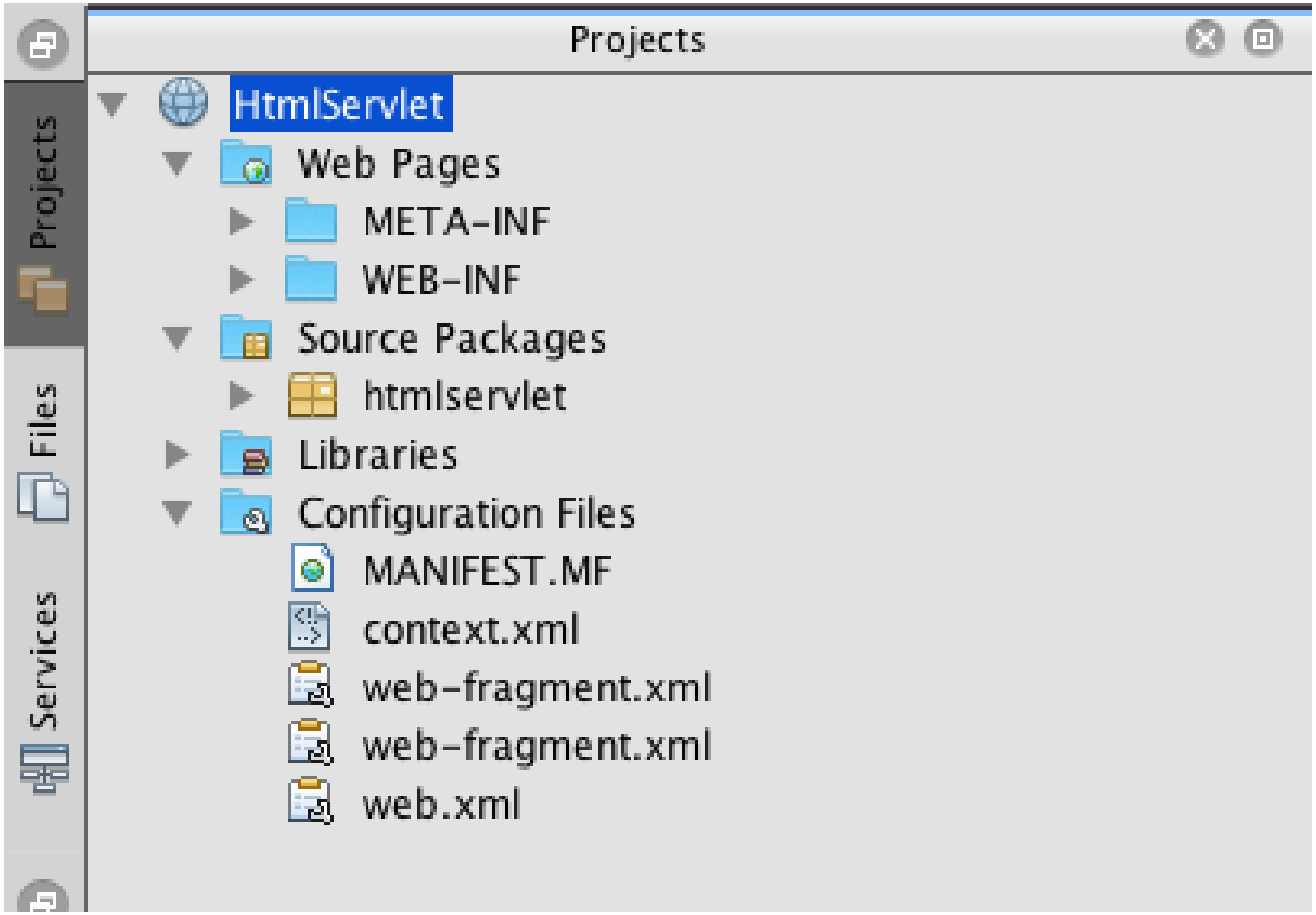
Exemple 2 : Servlet qui répond avec du html (suite)

```
    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response)
          throws ServletException, IOException {

       response.setContentType("text/html;charset=UTF-8");

       try(PrintWriter out = response.getWriter()) {
          out.println("<html>");
          out.println("<head>");
          out.println("<title>Servlet HtmlServlet</title>");
          out.println("</head>");
          out.println("<body>");
          out.println("Nom: <input type='text' name='nom'/><br/>");
          out.println("</body>");
          out.println("</html>");
       }
    }
}
```

Structure d'une App web sous Netbeans



Structure d'une App web sous Netbeans : fichier context.xml

- Ce fichier permet de définir le contexte de l'application.
- Le contexte est une partie de l'URL d'accès de l'application.
- Le contexte permet d'identifier l'application web au sein du serveur web.

Exemple :

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/HtmlServlet"/>
```

Structure d'une App web sous Netbeans : fichier web.xml

- Le fichier **`web.xml`** est le fichier de configuration de l'application web.
- Chaque servlet de l'application est déclarée dans ce fichier dans une section **`<servlet>`**.
- Une section **`<servlet>`** contient les balises **`<servlet-name>`** et **`<servlet-class>`** permettant respectivement de définir le nom du servlet et le chemin de la classe du servlet.
- Une section **`<servlet>`** peut contenir une sous section **`<init-param>`** pour définir des paramètres d'initialisations pour le servlet.

Structure d'une App web sous Netbeans : fichier web.xml

- Une sous section **<init-param>** contient les balises **<param-name>** et **<param-value>** permettant respectivement de définir le nom et la valeur du paramètre.
- Le fichier **web.xml** contient également une section **<servlet-mapping>** pour associer une URL au servlet.
- Une section **<servlet-mapping>** contient les balises **<servlet-name>** et **<url-pattern>** permettant respectivement de spécifier le nom du servlet et son url d'accès relative au contexte de l'application.

Exemple de fichier web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
   ...
   <servlet>
      <servlet-name>HtmlServlet</servlet-name>
      <servlet-class>htmlservlet.HtmlServlet</servlet-class>

      <init-param>
         <param-name>nbLignes</param-name>
         <param-value>5</param-value>
      </init-param>
   </servlet>

   <servlet-mapping>
      <servlet-name>HtmlServlet</servlet-name>
      <url-pattern>/HtmlServlet</url-pattern>
   </servlet-mapping>
</web-app>
```

Déclarer un servlet avec les annotations

- Le servlet peut être déclaré dans le fichier **web.xml** dans les sections **<servlet>** et **<servlet-mapping>**.
- Le servlet peut également être déclaré à l'aide d'annotations placées juste avant la déclaration de sa classe.

Exemple :

```
@WebServlet(
        name = "HtmlServlet",
        urlPatterns = {"/HtmlServlet", "/HServlet"},

        initParams = {
            @WebInitParam( name = "nbLignes", value = "5" ),
            @WebInitParam( name = "nbRow", value = "45" )
        }
)
public class HtmlServlet extends HttpServlet {
    ...
}
```

URL d'accès à un servlet

- L'URL d'accès à un servlet est composée des éléments suivants :
  1. protocole http ;
  2. nom ou adresse ip du serveur
  3. numéro de port (présent uniquement lorsqu'il est différent de 80)
  4. contexte de l'application
  5. URL pattern du servlet

- Exemple :

  `http://localhost:8080/HtmlServlet/HtmlServlet`